

f1-llm — Como os testes estão funcionando

Relatório técnico de status

Setembro de 2026

Resumo

O projeto **f1-llm** está sendo construído seguindo o ciclo de TDD (test-driven development) descrito na metodologia do Matt Pocock: fatias verticais, uma de cada vez, nunca “todos os testes primeiro, implementação depois”. Até agora, 2 das 7 ferramentas (*tools*) planejadas foram implementadas de ponta a ponta — `get_session_results` e `get_lap_times` — somando **14 testes**, todos passando: **10 rápidos** (sem rede) e **4 de integração** (dados reais da FastF1).

1 Metodologia: TDD em fatias verticais

O ciclo seguido para cada pedaço de funcionalidade é sempre o mesmo, em quatro passos:

1. **Confirmar o *seam*** (a fronteira pública que vai ser testada) antes de escrever qualquer teste.
2. **Red** — escrever um teste que falha, porque a implementação ainda não existe.
3. **Green** — escrever o mínimo de código necessário pra aquele teste passar, nada além disso.
4. Repetir para o próximo *seam*, um de cada vez.

Por que não “todos os testes antes”? A própria metodologia chama isso de *horizontal slicing* e trata como antipadrão: escrever a suíte inteira antes de qualquer código trava a estrutura da implementação antes de ela existir de verdade, e os testes acabam validando um comportamento *imaginado* em vez de aprendido ciclo a ciclo. Um exemplo real aconteceu neste projeto: só percebemos que `load_results` precisava de um valor padrão (`None`) depois de já termos escrito o segundo teste de validação — se tivéssemos escrito os quatro testes de uma vez, essa decisão teria sido um palpite, não uma consequência do ciclo.

A refatoração (por exemplo, mover `SessionDataUnavailable` para um módulo `errors.py` compartilhado) acontece *entre* ciclos, nunca misturada dentro de um passo red→green.

2 Duas categorias de teste

O projeto separa claramente testes rápidos de testes que dependem de rede/dados reais, usando um marcador do pytest configurado em `pyproject.toml`:

```
[tool.pytest.ini_options]
markers = [
    "integration: hits real external services (FastF1 network/cache);
        excluded by default.",
]
addopts = "-m 'not integration'"
```

	Suíte rápida	Suíte de integração
Comando	<code>uv run pytest</code>	<code>uv run pytest -m integration</code>
Depende de rede	Não	Sim, na primeira execução
Velocidade	<0,3s para os 10 testes	Minutos na 1ª vez; segundos depois (cache)
O que valida	Regras de negócio isoladas	A integração real com a FastF1
Uso típico	Rodar a cada alteração	Rodar antes de um commit importante

Dados baixados da FastF1 ficam em cache local (`.fastf1cache/`, fora do controle de versão), então a segunda execução da suíte de integração é rápida.

3 Estratégia de mock: só na fronteira do sistema

A regra seguida é simples: **nunca mockamos código nosso**, só a fronteira externa — neste caso, a biblioteca FastF1. Isso é feito por **injeção de dependência**: cada função de tool recebe um parâmetro `load_results/load_laps` que, em produção, aponta para o módulo `fastf1_client.py`, mas que nos testes rápidos recebe uma função falsa e determinística:

```
result = get_session_results(
    year=2023,
    event="Bahrain",
    session_type="Race",
    load_results=lambda year, event, session_type: raw_rows,
)
```

O próprio `fastf1_client.py` é uma exceção a essa regra — ele é literalmente a fronteira, então não há nada mais externo pra mockar. Por isso ele é validado com **dados reais**, na suíte de integração, usando uma sessão histórica estável e conhecida (o GP de Abu Dhabi de 2021) como fixture.

4 Cobertura atual

4.1 `get_session_results` — 4 testes rápidos + 2 de integração

Teste	Arquivo	Status
Rejeita <code>session_type</code> não suportado	<code>test_session_results.py</code>	PASS
Rejeita <code>year</code> fora do intervalo (< 2018)	<code>test_session_results.py</code>	PASS
Transforma dados brutos em <code>DriverResult</code>	<code>test_session_results.py</code>	PASS
Reporta “não encontrado” sem inventar dados	<code>test_session_results.py</code>	PASS
Carrega resultados reais (Abu Dhabi 2021)	<code>test_fastf1_client.py</code>	PASS
Detecta sessão que não existiu naquele ano	<code>test_fastf1_client.py</code>	PASS

4.2 get_lap_times — 6 testes rápidos + 2 de integração

Teste	Arquivo	Status
Rejeita <code>session_type</code> não suportado	<code>test_lap_times.py</code>	PASS
Rejeita <code>year</code> fora do intervalo	<code>test_lap_times.py</code>	PASS
Rejeita lista de <code>drivers</code> vazia	<code>test_lap_times.py</code>	PASS
Transforma voltas (incl. volta sem tempo registrado)	<code>test_lap_times.py</code>	PASS
Reporta “não encontrado”	<code>test_lap_times.py</code>	PASS
Constrói o gráfico Plotly (função pura, sem I/O)	<code>test_lap_times.py</code>	PASS
Carrega voltas reais filtradas por piloto	<code>test_fastf1_client.py</code>	PASS
Detecta quando nenhum piloto pedido aparece	<code>test_fastf1_client.py</code>	PASS

Detalhe de design: a construção do gráfico (`build_lap_times_chart`) foi separada da busca de dados de propósito. Como é uma função pura (sem chamadas externas), o teste não precisa de nenhum mock — basta passar uma lista conhecida de voltas e comparar a figura Plotly gerada.

5 Uma descoberta real do processo

Vale registrar porque ilustra o valor do teste de integração: a primeira versão do teste de “sessão não encontrada” usava um nome de evento inventado ("Not A Real Grand Prix"), esperando que a FastF1 falhasse. Ela não falhou — a biblioteca faz *fuzzy matching* de nomes de evento e silenciosamente “corrigiu” a entrada para "Monaco Grand Prix".

```
WARNING Correcting user input 'Not A Real Grand Prix' to 'Monaco Grand Prix'
```

O fixture foi ajustado para um cenário que realmente não existe: pedir uma sessão **Sprint** do GP do Bahrein de 2018, ano em que esse formato de sessão nem tinha sido criado. Esse tipo de comportamento inesperado da dependência externa só aparece testando contra a coisa real — é exatamente o motivo de mantermos a suíte de integração, mesmo sendo mais lenta.

6 Como rodar

```
# suite rapida, roda a cada alteracao (sem rede)
uv run pytest

# com detalhe de cada teste
uv run pytest -v

# suite de integracao, dados reais da FastF1 (mais lenta na 1a vez)
uv run pytest -m integration -v
```

7 Próximos passos

Restam 5 tools no escopo do v1, cada uma repetindo o mesmo ciclo (validação → transformação mockada → não encontrado → fronteira real):

- `compare_telemetry`

- `get_championship_standings`
- `plot_position_changes`
- `get_race_pace`
- `get_tyre_strategy`